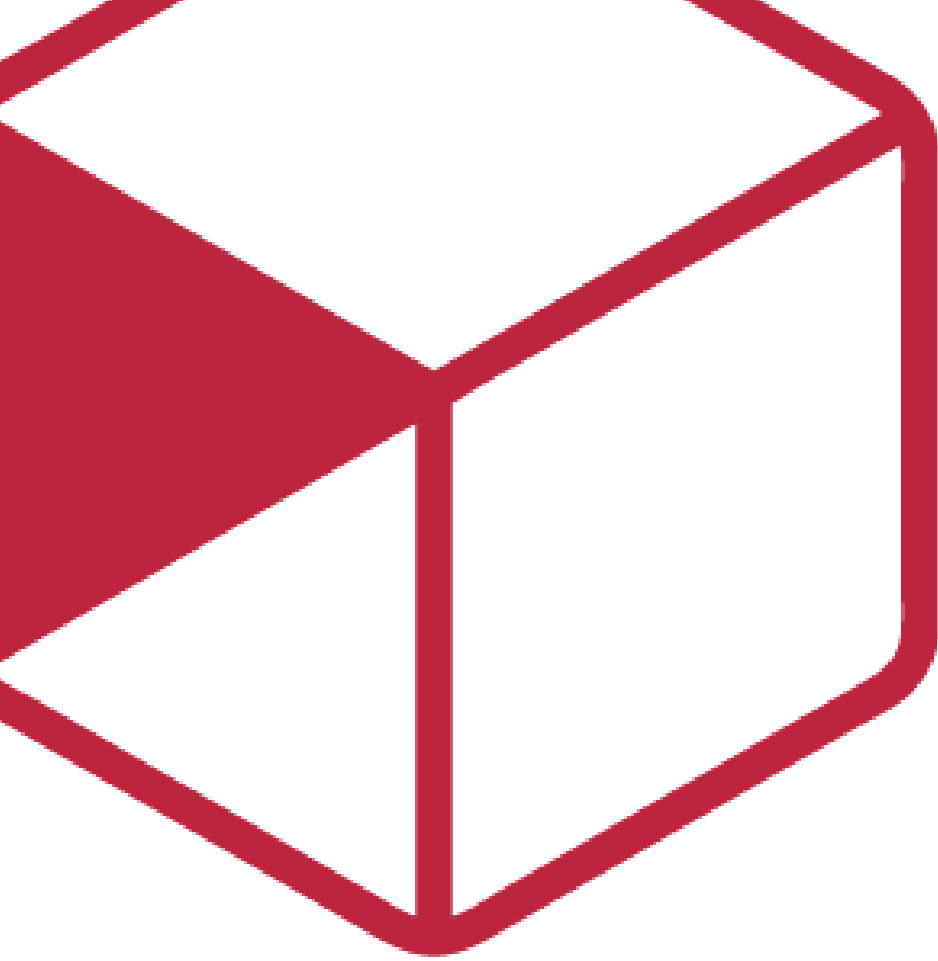




Diseño de la solución

Requisito R16A-RE-FU-XXX.

FORMATO	Arquitectura
PROYECTO	R16 - Adquisiciones
REFERENCIA	AUI- FOR-01
VERSIÓN	1.0
FECHA	16/06/2026
AUTOR	Alan Fernandez Garcia
REVISOR	Valdemar Farina Sanchez

Contenido

Importante	3
Introducción	4
1. Propósito del documento	4
2. Alcance	4
Específicamente incluye:	4
No se consideran:	5
**En caso de aplicar colocar lista de consideraciones que no aplicaran	5
Visión general del diseño	6
1. Objetivo técnico	6
2. Componentes involucrados	6
Diseño funcional detallado	7
1. Carga inicial y filtrado del historial	7
2. Criterios de aceptación del requisito	8
3. Reglas técnicas aplicadas	9
Diseño de componentes	10
1. Diagramas	10
Impacto Técnico	11
1. Impacto en código existente	11
2. Impacto en modelos por actualización de modelos	11
Manejo de Errores y Excepciones	12
Estrategia de Pruebas (Diseño de las pruebas)	12
1. Pruebas funcionales (Criterios de Aceptación en DEV)	13
2. Pruebas técnicas(unitarias e integración)	13
a. Unitarias	13
b. Pruebas de integración	13
3. Casos críticos	13
Control de versiones	14



Importante

Posterior a este diseño, ¿cómo saber si el diseño de la solución al requisito está completo para que el programador inicie con el desarrollo?

Hazte estas preguntas rápidas:

- ¿El programador sabe qué flujo implementar?
- ¿Sabe qué pasa si algo falla?
- ¿Sabe qué reglas no puede romper?
- ¿Sabe qué pruebas debe pasar?
- ¿Sabe dónde impacta?

Nota: Este documento es una propuesta basada en el estándar IEEE 1016 “Software Design Description” que va permitir abarcar los puntos más importantes para el diseño del requisito que se está trabajando. Se debe administrar el tiempo que se tiene de diseño para que se complete de la mejor manera considerando todos los detalles técnicos.



Introducción

1. Propósito del documento

El propósito de este documento es definir el diseño de la solución técnica para el requisito **R16M-RE-FU-042 (Historial de Notificaciones)**, describiendo los componentes de Angular, la gestión de estado con NgRx, la arquitectura de carpetas y los contratos de servicios necesarios para su implementación.

Nota: Este documento se enfoca exclusivamente en el diseño de la solución en el **Front-end**, no redefine requisitos funcionales ni lógica del **Back-end**.

2. Alcance

Este análisis cubre únicamente el requisito antes mencionado y considera:

- Componentes involucrados
- Ciclo de vida del estado con NgRx (Actions, Reducers, Selectors, Effects).
- Integración con APIs con Swagger.
- Estrategia de detección de cambios y optimización de rendimiento.
- Ajustes en flujos existentes
- Impacto en proyectos o funcionalidades existentes
- Flujos técnicos
- Interfaces y contratos
- Manejo de errores
- Consideraciones técnicas específicas

Específicamente incluye:

******Listado de tareas o acciones a implementar a alto nivel

- Creación de un módulo contenedor (Feature Module) con lazy loading.
- Implementación de una tabla reactiva con paginación y filtros.
- Configuración de la persistencia de filtros en la URL (Query Parameters).
- Integración de NgRx para el manejo del listado y marcas de lectura.



No se consideran:****En caso de aplicar colocar lista de consideraciones que no aplicaran**

- El diseño de los servicios tipo REST (se asume que el Back-end ya expone los endpoints necesarios).
- Estilos CSS/SASS globales (se utilizarán las clases utilitarias de la librería de UI existente).

Por ejemplo: El diseño aplica únicamente a la eliminación y marcado de correos como spam...
Debe quedar claro **hasta dónde llega** este documento.



Visión general del diseño

1. Objetivo técnico

Descripción clara del objetivo técnico del diseño (qué busca lograr la solución desde el punto de vista técnico), por ejemplo:

“Definir la forma en que el sistema permitirá la eliminación y marcado de correos como spam, garantizando persistencia, seguridad y trazabilidad.”

2. Componentes involucrados

Identificar **componentes reales** del sistema que participan e indicar la responsabilidad de cada uno.

Aplicativo	Componente	Responsabilidad	Ubicación
	UI / Frontend	Renderizar pantalla con comportamiento	
	NGRX(selector, reducer, action)	Gestionar el estado de un módulo de la aplicación	
	Componente reutilizable de librería	Proporcionar comportamiento definido y reutilizable	
	**Se pueden añadir más conforme se necesite	**	



Diseño funcional detallado

Descripción paso a paso del flujo técnico con orden lógico (**si alguien más lo lee, pueda dibujarlo en un diagrama de flujo**). Se ponen tantos flujos como se hayan identificado.

Por ejemplo:

1. Carga inicial y filtrado del historial

1. El usuario ingresa a la ruta /notificaciones.
2. El Router de Angular carga el componente contenedor NotificationHistoryContainer.
3. El contenedor obtiene los parámetros de búsqueda (paginación, filtros, ordenamiento, etc.).
4. El contenedor despacha la acción:
5. [Notifications] Load History
6. Un NgRx Effect escucha esta acción y se ejecuta.
7. El Effect llama al servicio de API:
8. GET /api/v1/notifications?page=1...
9. El Back-end procesa la solicitud y devuelve la información en formato JSON.
10. El Effect recibe la respuesta exitosa.
11. El Effect despacha una nueva acción:
12. [Notifications] Load History Success
13. El Reducer recibe la acción de éxito y actualiza el Store:
14. loading = false
15. entities = datos recibidos
16. error = null
17. Los Selectors detectan que el estado cambió y emiten los nuevos datos mediante Observables.
18. El componente NotificationHistoryContainer recibe automáticamente los datos utilizando el async pipe.
19. El contenedor envía los datos al componente de presentación mediante @Input().
20. El componente NotificationTable recibe los datos.
21. Debido a que utiliza ChangeDetectionStrategy.OnPush, Angular detecta el cambio de referencia en el Input.
22. La tabla se vuelve a renderizar mostrando el historial de notificaciones actualizado.



2. Criterios de aceptación del requisito

Realizar un mapeo explícito de los criterios de aceptación (CA) que demuestre que TODOS están cubiertos.

Ejemplo:

CA	Descripción	Estado	Justificación
CA-1	<i>El usuario debe poder filtrar por notificaciones "Leídas" y "No Leídas".</i> → Cubierto en Flujo 1 usando los Selectors filtrados.	Cubierto ▾	
CA-2	La paginación debe mantener los filtros activos en la URL para poder compartir el enlace. → Cubierto en Container mediante la sincronización con <i>ActivatedRoute.queryParams</i>.	Cubierto ▾	
CA-3	Al marcar como leída, el contador global de notificaciones del Header debe actualizarse. → <i>PENDIENTE: Requiere conectar este módulo con el Store Global (AppStore) o SharedStore del Header.</i>	Cubierto ▾	



3. Reglas técnicas aplicadas

Reglas necesarias para implementar el requisito correctamente (validaciones, restricciones).

NO son reglas de negocio, son reglas de implementación.

Ejemplo:

Regla	Descripción
RT-01	Todos los componentes de presentación (NotificationTable, NotificationFilters) DEBEN usar changeDetection: ChangeDetectionStrategy.OnPush.
RT-02	Queda estrictamente prohibido usar .subscribe() dentro del Container para pasar datos a los componentes hijos. Se debe usar el async pipe directamente en el template HTML (notificaciones\$ async).
RT-03	Toda la data proveniente de la API debe estar fuertemente tipada mediante una interfaz estricta (ej. export interface NotificationItem { id: string; title: string; date: string; isRead: boolean; }). No se permite el uso del tipo any.



Diseño de componentes

1. Diagramas

Diagramas incluidos en este diseño:

- Diagramas de secuencia y arquitectura Angular
- Flujo de interacción: Carga y filtrado con NgRx
- Estructura del Store del Módulo
- Diagrama de secuencia (involucrando los componentes identificados)

Los diagramas deben reflejar únicamente el funcionamiento del requisito actual.

Impacto Técnico

1. Impacto en código existente

Indicar cuales son los componentes a modificar o componentes nuevos (qué se toca y qué no)

Por ejemplo:

Modificaciones: Se debe alterar el archivo `app-routing.module.ts` para registrar la nueva ruta `/notificaciones` mediante Lazy Loading.

Componentes nuevos: Creación del directorio completo `features/notifications/` con sus subcarpetas de estado, componentes y servicios.

**Agregar archivo, línea de código o captura que permita ejecutarlo a alguien más sin saber todo el contexto

2. Impacto en modelos por actualización de modelos

Indicar cuáles serán los nuevos estados, actualizaciones de registros, etc.

Por ejemplo:

- Actualización del estado del correo se agregó nueva propiedad
- Se cambio el tipo de dato de un campo



Manejo de Errores y Excepciones

Qué errores se esperan y cómo debe responder el sistema. Por ejemplo:

Escenario	Comportamiento esperado
Error 500 / Timeout de API	El Effect captura la falla, cambia el estado loading: false, mantiene la tabla vacía y dispara el NotificationToastService mostrando un mensaje Toast rojo: "No se pudo cargar el historial. Reintente más tarde."
Error 403 (Sesión Expirada)	El HttpInterceptor global de la aplicación interceptará el código de error, limpiará el localStorage y redirigirá al usuario automáticamente al /login.
Petición repetida (Double Click)	Si el usuario presiona repetidamente el botón de cambiar página, el Effect debe usar el operador switchMap de RxJS para cancelar automáticamente la petición HTTP anterior que esté en vuelo y procesar únicamente la última.



Estrategia de Pruebas (Diseño de las pruebas)

1. Pruebas funcionales (Criterios de Aceptación en DEV)

- Verificar que al cambiar de página a la número 3, la URL del navegador se actualice a ?page=3.
- Verificar que al dar clic en "Marcar como leída", el renglón correspondiente cambie su opacidad visual de forma inmediata.

2. Pruebas técnicas (unitarias e integración)

Escribir las pruebas unitarias y de integración (las que usualmente hace el Programador). Por ejemplo:

a. Unitarias

- Verificar que el componente NotificationTable emita correctamente el @Output() con el ID correcto al hacer clic en el botón de lectura.
- Asegurar que los Selectors de NgRx devuelvan la información filtrada correctamente sin mutar el estado original.

b. Pruebas de integración

- Probar el NotificationEffects utilizando provideMockActions para asegurar que ante una acción LoadHistory, se dispare un LoadHistorySuccess tras la respuesta del servicio simulado (Mock).

3. Casos críticos

- **Flujo con Red Inestable:** Simular mediante las herramientas de desarrollador del navegador (Network Throttling - Slow 3G) un retraso en la petición de marcado como leída para garantizar que el indicador de carga (Spinner) de la fila se mantenga activo bloqueando interacciones secundarias del usuario en esa misma fila.



Control de versiones

Versión	Fecha	Autor	Tipo de Cambio	Descripción del cambio	Aprobó
1.0	06/03/25	Roberto Baez ...	Creación ▾	Se crea plantilla genérica	Rose Ríos ...
1.1	16/06/26	Alan Fernand...	Creación ▾	Se crea plantilla genérica para DIS	

